

Portfolio Project Brief

Software Testing and Design Patterns

Advanced Software Engineering - Module 7

How to use this brief:

Read this document in full and bring any questions you have on the assessment to either a workshop or the next 1:1 with your instructor. **Advance in your project journey by completing milestones**, which are progress checks on tasks that build to your overall output. Milestone submission and reviews will be part of your workshop and independent work. **Self-assess your progress** against the assessment criteria table while completing your project.

Module Title	Software Testing and Design Patterns		
Format	Technical Report	Deadline	On MyMultiverse
Length	4,000 words	Feedback Due	Within 20 working days of submission date
Project Title	Software Quality: Design and Testing Practice		
Project Learning Outcomes	1. Analyse the software development lifecycle to identify where quality is built in or absent, establishing a data-backed case for improvement.(K21) 2. Evaluate architectural characteristics, including coupling and design patterns, to justify improvements that increase system testability(K22, K23) 3. Implement design changes and deploy an automated testing strategy to evaluate quality across multiple stages of the development lifecycle. (S21) 4. Present technical design decisions and quality outcomes to both technical and non-technical stakeholders to demonstrate professional collaboration.(K8, B1)		
Project Milestones	Milestone 1 (by Week 3): Produce a current-state analysis of software quality practice in your organisation. Locate your development practice within the SDLC, describe how design decisions are currently made, and assess where testing activity sits relative to the development process. Identify at least two specific quality risks or gaps with business impact. Milestone 2 (by Week 6): Evaluate the design and architectural characteristics of a specific software system. Analyse design patterns present or absent, assess coupling and structural decisions, and evaluate their implications for quality and testability. Produce an architectural		

	decision record documenting at least two justified improvements, with testability implications and an outline of the testing approach for Milestone 3. Milestone 3 (by Week 10): Implement the design improvements and execute a testing strategy that responds to those decisions. Apply automated testing, black-box and white-box techniques, and boundary analysis. Integrate with CI/CD tooling. Produce the final report, including a critical evaluation of quality impact and evidence of stakeholder communication throughout the project.
--	--

How to use this brief:

Read this document in full and bring any questions you have on the assessment to either a workshop or the next 1:1 with your instructor. Advance in your project journey by completing milestones, which are progress checks on tasks that build to your overall output. Milestone submission and reviews will be part of your workshop and independent work. Self-assess your progress against the assessment criteria table while completing your project.

Your grade reflects the depth and quality of your analysis, not the size or sophistication of the system you work on or the organisation you work in. A restricted scope, a heavily governed environment, or a team with limited testing infrastructure is not a disadvantage - it is the context for your analysis. The strongest submissions use their organisational context as the subject of their diagnosis, not as a justification for its limits. An apprentice working on a single service within a large, constrained organisation and an apprentice with broad access to a small team's entire codebase are being assessed against the same standard: the quality of their thinking about what they find.

When scoping your project, choose a system or component that you have genuine access to and real knowledge of. A narrowly scoped project analysed with depth and honesty will always outperform a broadly scoped project treated superficially. If you are unsure whether your chosen scope is appropriate, bring it to your Session A group coaching session before you begin.

Final Project Output

Your Technical Report will document two connected phases of professional software engineering practice. In the first phase, you will analyse a software system's architecture, evaluate the design decisions made - including the application of design patterns, structural coupling, and architectural choices - and produce a documented rationale for recommended improvements. In the second phase, you will develop a testing strategy that responds directly to those design decisions, identifying what each decision makes testable, what it makes harder to test, and how testing should be distributed across the software development lifecycle.

The report will demonstrate that design and testing are not separate activities but connected disciplines: the design decisions you make determine the testing challenges you face. A well-reasoned, proportionate set of improvements supported by an honest evaluation of their quality impact will always outperform a broad but superficial treatment of the subject.

Suggested Report Structure

The structure below provides suggested guidance on content coverage and approximate word counts. You may adapt the section titles and organisation to best suit your project, provided all Learning Outcomes are evidenced in your submission.

Section	Content covered
1. Quality Diagnosis (500 words)	Organisation's current development practice located within the SDLC. Relationship between design practices, testing activity, and software quality outcomes. Identification of at least two specific quality gaps with business impact. The case for the improvements explored in the project. (LO1)
2. Design Evaluation and Architectural Decisions (1,100 words)	Analysis of the design and architectural characteristics of the chosen system. Evaluation of design patterns present or absent - creational, structural, behavioural. Assessment of coupling, structural decisions, and quality implications. Architectural decision record documenting at least two recommended improvements with testability implications and business justification. (LO2)
3. Testing Strategy and Implementation (1,100 words)	Testing strategy mapped to relevant SDLC stages. How design decisions from Section 2 shaped the testing approach. Test suite implementation - automated tests, black-box and white-box techniques, boundary analysis. CI/CD integration and tooling. Test results and coverage. Refinement of approach based on outcomes and evaluation of effectiveness. (LO3)
4. Stakeholder Communication and Professional Practice (700 words)	Communication of design decisions and quality outcomes to technical and non-technical stakeholders. Evidence of collaborative practice - peer review, design critique, test result presentation. Application of professional standards throughout the project. Impact of the work on the team or organisation. (LO4)
5. Critical Evaluation and Recommendations (600 words)	Critical evaluation of design decisions made - were they the right choices? Critical evaluation of the testing strategy - was it effective, what did it miss? Assessment of quality improvements achieved and gaps that remain. Recommendations for next steps and future iterations. Lessons learnt. (LO3, LO4)

How should I submit this Portfolio Project?

Your submission should be prepared as a single PDF file. Please make sure your work adheres to the MV referencing and style guide (see MyMultiverse). All work is checked for plagiarism using Turnitin.

Ensure that you meet the MV guidelines for academic integrity by considering the following:

- **Referencing:** When discussing someone else's work, clearly acknowledge the work by using in-text citations and a reference list adhering to the Harvard referencing style.
- **Word Count:** Any text exceeding the maximum word limit will not be marked. Text that adds to the word count includes all text except the reference list or appendices. All text in diagrams and in-text citations count towards the overall word limit.
- **Submission time:** All submissions must be made by mid-day on the specified deadline.

Assessment Rubric

Your assessment will be evaluated using the following rubric in relation to the Learning Outcomes. The grade reflects the depth and quality of your analysis and communication, not the sophistication of your organisation's technology or the scale of the system you have chosen.

LO	F	E	D	C	B	A
1. K21	Does not analyse current practice or fails to locate it within any recognisable SDLC framework.	Identifies basic SDLC stages but describes the organisation's practice without assessing quality or testing activity.	Describes current development practice and locates it within the SDLC. Outlines testing activity but without analysing quality implications or connecting gaps to business impact.	Explains the relationship between current design practices, testing activity, and software quality outcomes. Identifies specific quality gaps with reference to their business context and establishes a clear case for the improvements explored in the project.	Analyses the quality implications of current practice in depth, justifying identified gaps with evidence from the organisation and connecting them explicitly to specific SDLC stages. Demonstrates understanding of how design and testing decisions interact to produce quality outcomes.	Critically evaluates current practice against professional standards and emerging approaches - for example shift-left testing or architectural review practices. Synthesises an evidence-based quality diagnosis that produces original insight into the organisation's quality position and clearly motivates the project's scope and direction.
2. K22 K23	Does not evaluate design or architectural characteristics. Fails to identify any recognisable design patterns or quality implications.	Identifies design patterns by name but does not evaluate their architectural role, quality implications, or relevance to the organisational context.	Describes design patterns and architectural characteristics present in the system. Outlines quality or testability implications but does not justify recommendations against the business context or assess proportionality.	Explains the principles behind selected design patterns and their architectural role. Identifies quality and testability implications and recommends improvements with reference to business requirements and engineering principles.	Analyses how design decisions - including pattern selection, coupling, and structural choices - interact to affect software quality and testability. Justifies recommendations with context-specific evidence and demonstrates proportionality -	Critically evaluates the system's design against organisational quality requirements. Synthesises an architectural decision record that demonstrates sophisticated professional judgement - for example, identifying where a pattern introduces unnecessary complexity, where deferred decisions are strategically

					showing awareness of when simpler solutions are preferable to complex pattern implementations.	sound, or where competing design forces require explicit trade-off reasoning.
3. S21	Does not implement design improvements or execute automated tests. Fails to apply any recognisable testing technique or connect testing to the SDLC.	Produces a limited set of tests without demonstrating systematic technique selection or methodology awareness. Does not connect the testing approach to the design decisions documented in the project.	Creates and executes automated tests using at least one technique. Describes the testing approach but does not explicitly connect it to the design decisions made, evaluate its effectiveness, or demonstrate refinement based on outcomes.	Explains how the testing strategy responds to design decisions documented. Applies both black-box and white-box techniques, identifies critical functionalities and edge cases, integrates testing with CI/CD tooling, and reports results with coverage awareness.	Analyses testing outcomes and refines the approach based on results. Evaluates the effectiveness of techniques used, identifies limitations in coverage, and connects testing findings explicitly to the quality improvements sought - demonstrating the determination, refinement, and adaptation.	Critically evaluates the testing strategy as a whole - evaluating what it proved, what it missed, and what the results mean for software quality in the organisational context. Synthesises evidence-based conclusions that connect design decisions, testing outcomes, and quality impact in an original and professionally convincing way.
4. K8 B1	Does not document stakeholder communication or demonstrate professional standards. Shows no awareness of collaborative practice or team working.	Provides basic documentation. States that communication with stakeholders took place but provides no evidence of how design decisions or quality outcomes were conveyed, or how collaborative practice shaped the project.	Describes communication with at least one stakeholder group and identifies professional standards applied during the project. Does not explain how communication was adapted for different audiences or how collaboration influenced design or testing decisions.	Explains how design decisions and quality outcomes were communicated to both technical and non-technical stakeholders. Describes at least two collaborative practices - for example design critique - with reference to their impact on the project and evidence of	Analyses the effectiveness of stakeholder communication and collaborative practice, with specific examples of how feedback received influenced design or testing decisions. Demonstrates sustained commitment to quality throughout the project lifecycle and applies professional	Critically evaluates the communication and collaboration strategy as a whole - synthesising insights into how professional practice shaped project outcomes, what worked and what would be done differently, and how the apprentice's approach reflects and develops the professional standards expected of a senior software engineering practitioner.

				professional standards applied consistently.	standards with clear awareness of their purpose.	
--	--	--	--	--	--	--

Knowledge, Skills, and Behaviours (KSBs)

This project and the LOs will cover the following KSBs when you write up your portfolio:

K8	How teams work effectively to produce digital and technology solutions.
K21	How to operate at all stages of the software development life cycle and how each stage is applied in a range of contexts. For example, requirements analysis, design, development, testing, implementation.
K22	Principles of a range of development techniques, for each stage of the software development cycle that produce artefacts and the contexts in which they can be applied. For example UML, unit testing, programming, debugging, frameworks, architectures.
K23	Principles of a range of development methods and approaches and the contexts in which they can be applied. For example Scrum, Extreme Programming, Waterfall, Prince2, TDD.
S21	Determine, refine, adapt and use appropriate software engineering methods, approaches and techniques to evaluate software engineering project outcomes.
B1	Has a strong work ethic and commitment in order to meet the standards required.